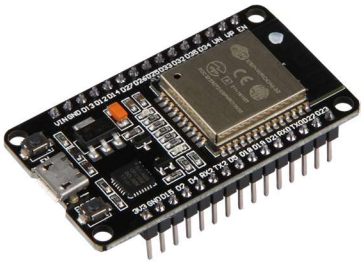
ESP32: ESP-NOW Wi-Fi Communications

by Henry Li

Introduction

You are expected to work in pairs for this lab experiment, as you will each act as the message sender and the other one as the receiver.



The **ESP32** series are low-cost, low-power system-on-chip (SoC) microcontroller with integrated Wi-Fi & Bluetooth (usually BLE, Bluetooth Low-Energy) developed by Espressif Systems. It is widely used for IoT and robotics projects. We will be using the Joy-It NodeMCU development kit, which has the ESP32-WROOM-32 SoC implemented on it.

ESP-NOW is a wireless communication protocol developed by Espressif Systems for their own SoC chips, including the ESP32 and ESP8266 series.

We will be using the well-known open-source IDE, **Visual Studio Code**, for ESP32 development here at Makers'. But of course, you may use any other IDEs if you wish to.

Espressif Systems developed their own extension, ESP-IDF, on VS Code dedicated for ESP32 development. But due to efficiency and compactibility factors, we will not be using ESP-IDF. Instead, we will be using something much more light-weight, efficient, and widely compatible called **PlatformIO**.

PlatformIO is a cross-platform, open-source ecosystem for embedded systems & IoT software development. It is commonly used on VS Code as an extension, and it contains an enoumous range of libraries, supporting most commonly used microcontroller boards like Arduino, ESPs, STMs, Raspbery Pis, Adafruit, ATmega, nRF52s, and so much more.

1. Getting Started – Build a simple circuit for testing

1.1. Build the circuit shown below for testing the basic sending and receiving commands with ESP-NOW.

Components

Component	Qty	GPIO Pin
Joy-It NodeMCU-ESP32 Development Kit	1	–
Light-Emitting Diode	1	IO13
220Ω Resistor	1	–
DIP 2-Pin PTM Switches	1	IO4
5V Power Supply (ignore if the ESP32 is connected to a PC)	(1)	VIN/ GND

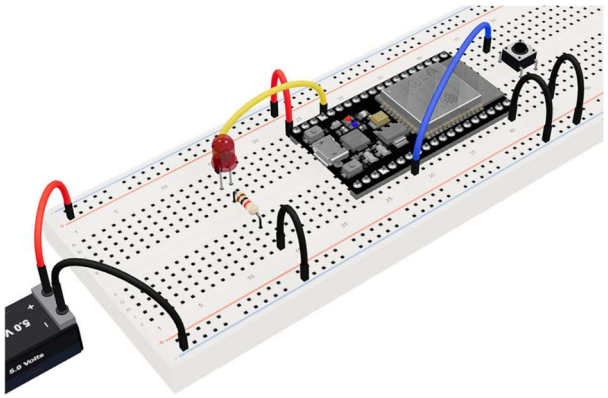


Fig.1 – Orthographic view of testing circuit. [1]

Circuit Visual Diagram

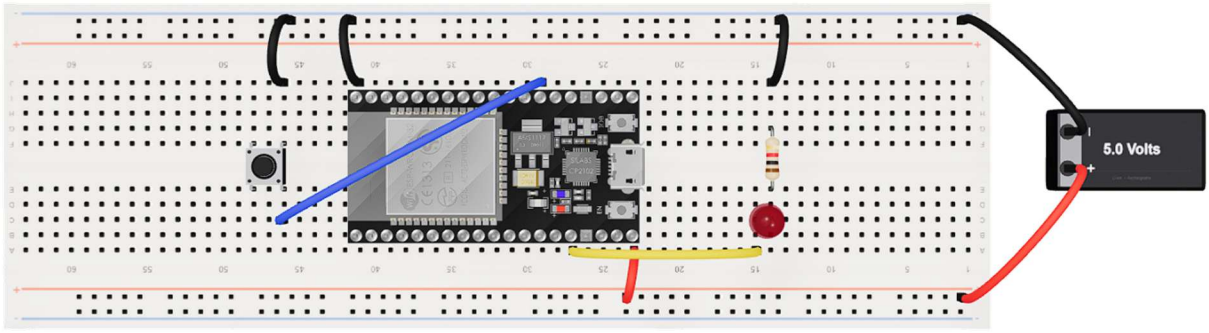


Fig.2 – Top view of testing circuit. [1]

2. Installing the Silicon Labs CP2102 Driver

(Please skip this section if you are working on a Shilling Lab PC)

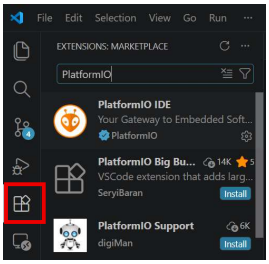
The CP210x series driver, developed by Silicon Labs, is required for connecting and uploading codes to an ESP32 microntroller.

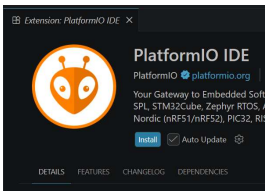
2.1. Go to https://www.silabs.com/software-and-tools/usb-to-uart-bridge-vcp-drivers

2.2. Go to the 'Download' tab, and click on the driver for the suitable OS. For Windows, I would recommend downloading the "**CP210x Windows Drivers**" from 2020, instead of the latest universal version, as the universal version don't seem to always work well with all Windows devices.

2.3. Once it's downloaded, run the setup files (For Windows, unzip the folder and run the Batch File).

3. Setting up your first PlatformIO project





3.1. Open **Visual Studio Code**.

3.2. Go to '**Extensions**' on the sidebar, and search for "**PlatformIO**".

3.3. Install the extension.

3.4. Then click on the '**PlatformIO**' icon on the sidebar, and clicking on **QUICK ACCESS >> PIO Home \ Open**, this will open up the homepage of PlatformIO.

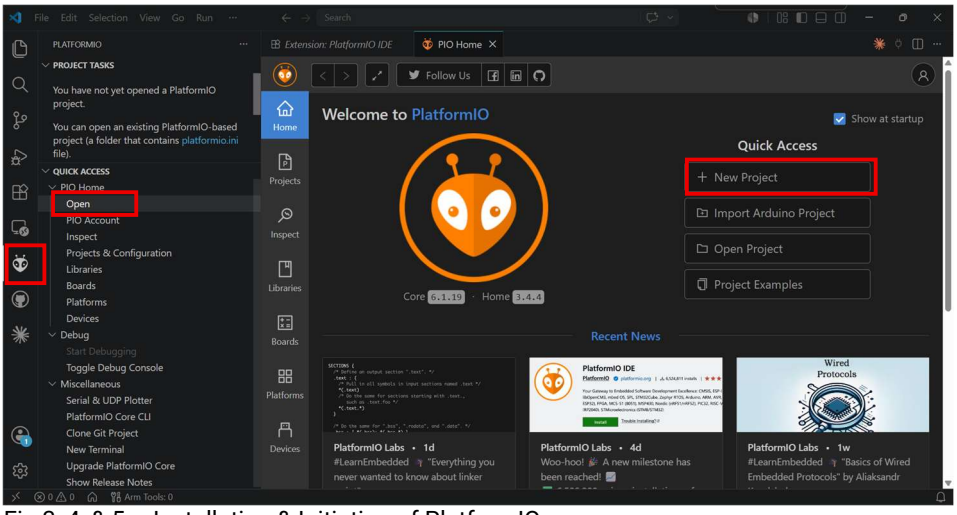


Fig.3, 4, & 5 – Installation & Initiation of PlatformIO.

3.5. You can then create a new project by clicking on '**New Project**'. The Project Wizard will then pop-up, and you can input the name of your project, and select the microcontroller board and its framework of this project.

For this project, select "**Espressif ESP32 Dev Module**" as the board, and the "**Arduino**" framework. I would highly recommend that the project location is set to default. If you are unsure about where your default location is, simply hover the cursor over the circled question mark next to the option box.

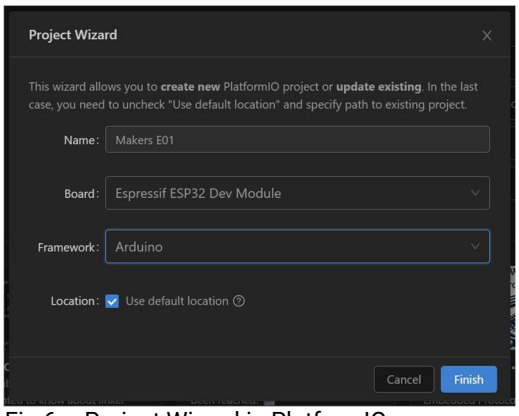


Fig.6 – Project Wizard in PlatformIO.

3.6. Once the project is initialised, open '**src \ main.cpp**' and '**platformio.ini**'.
'**main.cpp**' is your source code file. This is where the full program goes, where '**platformio.ini**' is the initialising configuration file, where the system reads all the system platform, microcontroller board type, programming framework, and the required libraries information when the program is uploaded to the microcontroller.

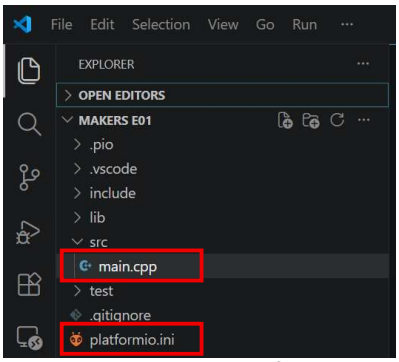


Fig.7 – Folder structure of a new ESP32 project with PlatformIO.

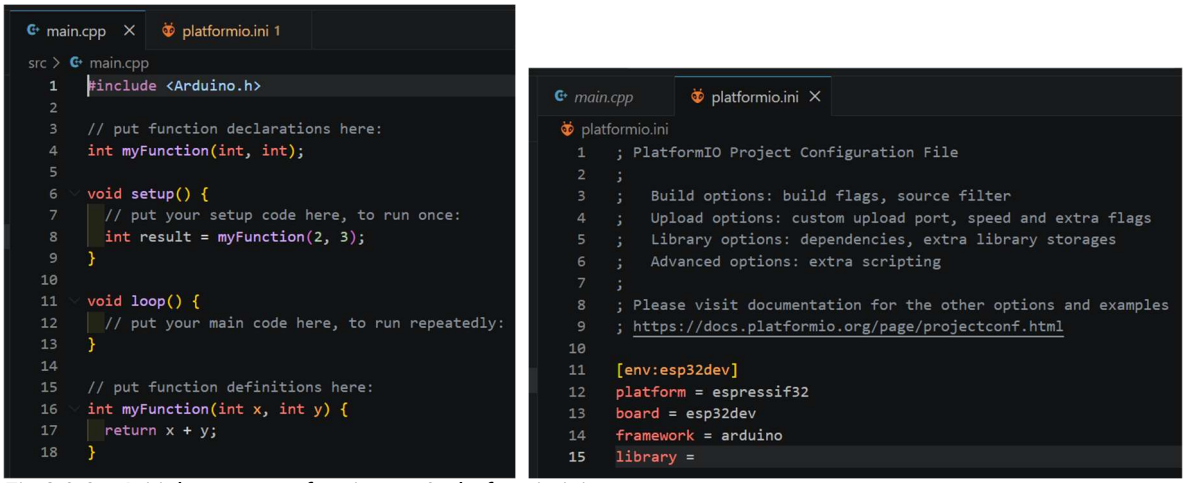


Fig.8 & 9 – Initial structure of main.cpp & platformio.ini.

4. Setting up ESP-NOW for wireless communications

(From this section onwards, we will be working in the '**main.cpp**' source code file unless stated otherwise)

4.1. Firstly, you need the peer device's MAC address in order to start coding. To get the MAC address of your device, write the following code into **void setup()**, then build and upload, and monitor the program. (See **Section 7** if you require any assistance)

Note: MAC addresses comes in the form of "0A, 1B, 2C...", which means you will need to manually add the "0x" prefix (as required by the coding environment) before each block of the MAC address.

```
#include <WiFi.h>

void setup() {
  Serial.begin(9600);
  delay(1000);
  WiFi.mode(WIFI_STA);
  Serial.println();
  Serial.println(WiFi.macAddress());
}
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SERIAL MONITOR
 *  Executing task: C:\Users\henry\.platformio\penv\Scripts\platformio.exe run --target upload --target monitor --environment esp32dev
Hard resetting via RTS pin...
--- Terminal on COM8 | 115200 8-N-1
--- Available filters and text transformations: debug, default, direct, esp32_exception_decoder, hexlify, log2file, nocontrol, printable, send_on_enter, time
--- More details at https://bit.ly/pio-monitor-filters
--- Quit: Ctrl+C | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H
ets Jul 29 2019 12:21:46

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1184
load:0x40078000,len:13232
load:0x40080400,len:3028
entry 0x400805e4

C8:85:41:C4:31:58
```

Fig.10 – Location of device MAC address.

4.2. Now clear **void setup()**, and to start with, include the following libraries at the top of the file:

```
#include <Arduino.h>
#include <WiFi.h>
#include <esp_now.h>
#include <Wire.h>
```

4.3. Then, define the GPIO pins and the Wi-Fi channel as follows: (you may change the channel to prevent any interceptions between different sender-receiver pairs)

```
#define PIN_SW1  4
#define PIN_LED  13
#define WIFI_CHANNEL 1
```

4.4. We will then set up the data structure ("**sendData**" for the sender, and "**receivedData**" for the receiver) as well as the MAC address of the peer device (i.e. put the MAC of the receiver ESP into the sender's code).

```
typedef struct struct_message {
  const char* message;
} struct_message;

struct_message sendData;

uint8_t peerMAC[] = {0x00, 0x00, 0x00, 0x00, 0x00, 0x00};
esp_now_peer_info_t  peerInfo;
```

From this step onwards until **Section 7**, the sender should carry on to the next section (**Section 5**) for coding the message sender, and the receiver should skip to **Section 6** for coding the message receiver.

5. The Message Sender

(If you are the receiver, skip this section and carry on from Section 6)

5.1. Firstly, before **void setup()**, write the following code to define the data delivery status, which will be used later on during the device setup.

```
void DataDeliveryStat(const uint8_t *peerMAC, esp_now_send_status_t espNOWstatus) {
  Serial.print("Packet Send Status: \t");
  Serial.println(espNOWstatus == ESP_NOW_SEND_SUCCESS ? "Delivery Successful" : "Delivery
Failed");
}
```

5.2. We will then start setting up the device. In **void setup()**, write these lines in to set up serial monitor, the 2 GPIO pins, and to initialize ESP-NOW.

```
  Serial.begin(115200);

  pinMode(PIN_SW1, INPUT_PULLUP);
  pinMode(PIN_LED, OUTPUT);
  digitalWrite(PIN_LED, LOW);

  WiFi.mode(WIFI_STA);

  if (esp_now_init() != ESP_OK) {
    Serial.println("ESP-NOW Initialization Failed.");
    delay(1000);
    return;
  } else if (esp_now_init() == ESP_OK) {
    Serial.println("ESP-NOW Initialization Complete.");
  }
```

5.3. Finally in the setup function, write these codes to tell your ESP to register the receiver's ESP as a connected receiving peer device and get the status information with the serial monitor.

```
  esp_now_register_send_cb(DataDeliveryStat);
  memcpy(peerInfo.peer_addr, peerMAC, sizeof(peerMAC));
  peerInfo.channel = WIFI_CHANNEL;
  peerInfo.encrypt = false;

  if (esp_now_add_peer(&peerInfo) != ESP_OK) {
    Serial.println("ESP-NOW Connection Failed. Device Not Added.");
    delay(1000);
    return;
  } else if (esp_now_add_peer(&peerInfo) == ESP_OK) {
    Serial.println("Receiver Device Connected via ESP-NOW.");
  }

  Serial.print("Sender Device MAC Address: " + WiFi.macAddress());
  Serial.print(" | Receiver Device MAC Address: " + String(peerMAC[0], HEX) + ":" +
String(peerMAC[1], HEX) + ":" + String(peerMAC[2], HEX) + ":" + String(peerMAC[3], HEX) + ":"
+ String(peerMAC[4], HEX) + ":" + String(peerMAC[5], HEX));
  Serial.println(" | Wi-Fi Channel: " + String(WIFI_CHANNEL));
```

5.4. We can now move on **void loop()**, where the program actually loops the commands. Firstly, write this line of code to declare the actual content of your message. It doesn't have to be a single word, it can be anything! (as long as they all goes into the quotation marks!)

```
  const char* message = "Hello!";
```

5.5. We will then write these codes create an 'IF statement' after declaring the message, to identify if the button is pushed.
If yes, the system will switch on the LED, and send the message to the peer device, or if otherwise, then the system will make sure the LED is off, and return to the top of this IF statement.

```
  if (digitalRead(PIN_SW1) == LOW) {
    digitalWrite(PIN_LED, HIGH);
```

```
    Serial.println("Sending Message: \t" + String(message));

    sendData.message = message;
    esp_now_send(peerMAC, (uint8_t*) &sendData, sizeof(sendData));
    digitalWrite(PIN_LED, LOW);
    delay(500);
  } else {
    digitalWrite(PIN_LED, LOW);
    return;
  }
```

You have now built the message sender with ESP-NOW! Carry on to **Section 7** to build and upload your code onto the micrcontroller!

6. The Message Receiver

(If you are the sender, return to Section 5 or carry on to Section 7)

6.1. Firstly, before **void setup()**, write the following code to define the data receiving function, which will be used later on during the device setup.

```
void onDataReceive(const uint8_t* peerMAC, const uint8_t* data, const char* length) {
  if (length == sizeof(struct_message)) {
    memcpy(&receivedData, data, sizeof(receivedData));
    Serial.printf("Received Message: %s\n", receivedData.message);
  } else {
    Serial.printf("Unexpected Data Length: %d (expected %d)\n", length, sizeof(receivedData));
  }
}
```

6.2. We will then start setting up the device. In **void setup()**, write these lines in to set up serial monitor, the 2 GPIO pins, and to initialize ESP-NOW.

```
  Serial.begin(115200);

  pinMode(PIN_SW1, INPUT_PULLUP);
  pinMode(PIN_LED, OUTPUT);
  digitalWrite(PIN_LED, LOW);

  WiFi.mode(WIFI_STA);

  if (esp_now_init() != ESP_OK) {
    Serial.println("ESP-NOW Initialization Failed.");
    delay(1000);
    return;
  } else if (esp_now_init() == ESP_OK) {
    Serial.println("ESP-NOW Initialization Complete.");
  }
```

6.3. Finally in the setup function, write these codes to tell your ESP to register the receiver's ESP as a connected receiving peer device and get the status information with the serial monitor.

```
  esp_now_register_recv_cb(onDataReceive);
  memcpy(peerInfo.peer_addr, peerMAC, sizeof(peerMAC));
  peerInfo.channel = WIFI_CHANNEL;
  peerInfo.encrypt = false;

  if (esp_now_add_peer(&peerInfo) != ESP_OK) {
    Serial.println("ESP-NOW Connection Failed. Device Not Added.");
    delay(1000);
    return;
  } else if (esp_now_add_peer(&peerInfo) == ESP_OK) {
    Serial.println("Sender Device Connected via ESP-NOW.");
  }
```

```
  Serial.print("Receiver Device MAC Address: " + WiFi.macAddress());
  Serial.print(" | Sender Device MAC Address: " + String(peerMAC[0], HEX) + ":" +
String(peerMAC[1], HEX) + ":" + String(peerMAC[2], HEX) + ":" + String(peerMAC[3], HEX) + ":"
+ String(peerMAC[4], HEX) + ":" + String(peerMAC[5], HEX));
  Serial.println(" | Wi-Fi Channel: " + String(WIFI_CHANNEL));

  Serial.println("Setup complete. Awaiting for data...");
```

6.4. We can now move on **void loop()**, where the program actually loops the commands, and this is much simpler than the sender. All you need to do is control the LED to flash when the message arrives!

```
  if (receivedData.message != NULL) {
    digitalWrite(PIN_LED, HIGH);
    delay(500);
    digitalWrite(PIN_LED, LOW);
    delay(50);
  }
```

You have now built the message receiver with ESP-NOW! Carry on to **Section 7** to build and upload your code onto the micrcontroller!

7. Uploading the program to the microcontroller

You're doing great if you have got to this stage! Now let's get this built and uploaded, so you can start sending some messages around the lab!

7.1. Firstly, you should find that there are some tiny icons on the status bar at the bottom of the window, or alternatively you can go and expand the PIO tab on the sidebar. **Don't forget to plug your ESP32 microncontroller into the PC before uploading!**

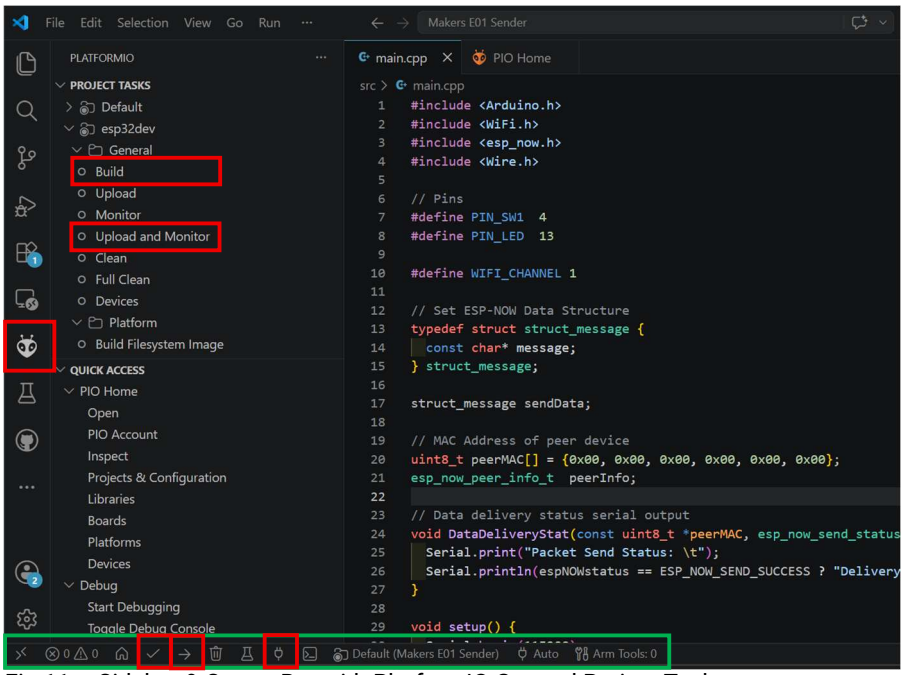


Fig.11 – Sidebar & Status Bar with PlatformIO General Project Tasks.

7.2. Now click on '**Project Tasks >> esp32dev >> General >> Build**' (sidebar), or the tick '✓' (status bar) and let the system verify, build and compile your program. It should say something like "**[SUCCESS] Took 10.23 seconds**". If the build failed or there are any errors occurred, seek for assistance from a committee member.

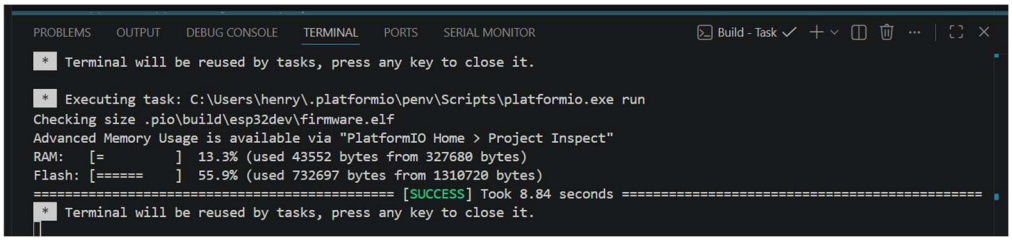


Fig.11 – Program built successfully.

7.3. Final finally, click on Upload and Monitor (sidebar), or the right arrow '➔' and the plug (status bar) to upload your program and open the serial monitor. Wait until the system has finished uploading the code and then you're ready to start messaging!

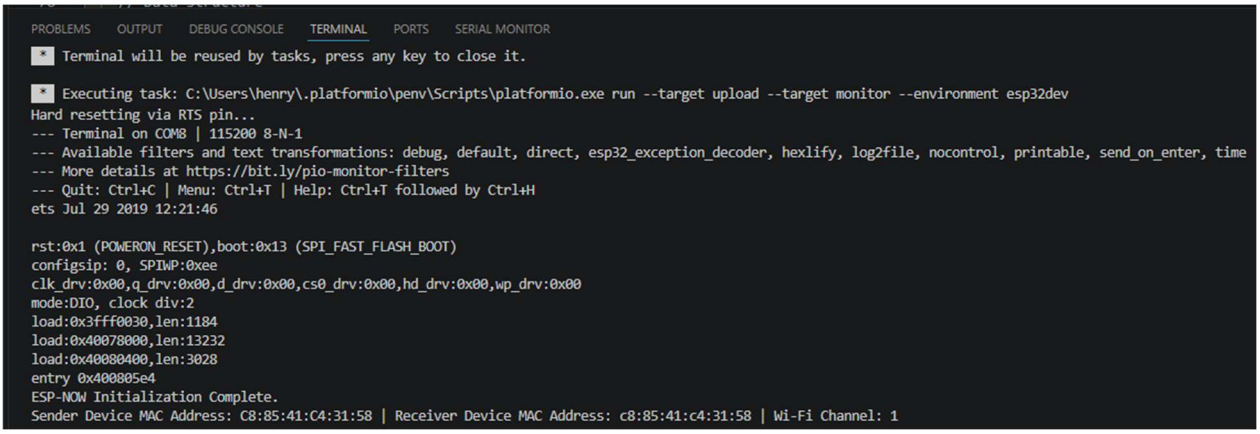


Fig.12 – Program uploaded onto ESP32 microcontroller.

8. Complete Codes [2]

- Sender Code
- Receiver Code

9. Something worth knowing…

Frameworks

For ESP32 development, PlatformIO offers both the Arduino and the official ESP-IDF frameworks. Both of them uses the C-family programming language as the primary language (Arduino uses C++ with simplified syntax, and ESP-IDF uses a mix of C and C++), and of course there are pros and cons to either of the frameworks.

For the **Arduino** Framework, although this is not really an industy-level standard framework, Arduino is easy and great for beginners to any embedded electronics. The framework uses the **setup()** and the **loop()** functions to run the full program, making it much easier for anyone to pick up. It also supports the use of community libraries, which allows you to directly make use of most generic **I2C & SPI components** in the market. But this does result in a **higher memory usage and larger binary size**, as well as a very limited control over the system parameters such as allocating specific buffer sizes, tweaking the partition table on the fly, or altering flash memory speeds.

Whereas for **ESP-IDF**, this is an **industrial-standard** level framework. With its native engine and direct access to the silicon, you are required to **directly design** the **full multi-threaded firmware** and program by directly accessing the registers and configuring the drivers with the use of **FreeRTOS**. This does mean that it allows you to rigorously optimize the full program, creating **low-latency interrupt handling**. Though, as ESP-IDF relies heavily on official Espressif drivers and components from their own Component Registry, you will occasionally find that you will need to manually port third-party libraries from C++ Arduino code to pure C/ESP-IDF drivers. We will come across using ESP-IDF later on in some advanced level lab experiments.

References

[1] H. Li. "MakersSoc E01 ESP-NOW Tutorial," Tinkered.ai, 28 June 2026. [Online]. Available: https://app.tinkered.ai/p/1105eeac-de5c-4bc8-a600-0037e4d9a5c9. [Accessed 29 June 2026].

[2] H. Li, c/o Royal Holloway Makers' Society. "RoyalHollowayMakersSoc/E01_ESP-NOW_Message_Comms: An ESP-NOW-based message communicator tutorial project with the use of two standard ESP32-WROOM-32 in Arduino Framework.," GitHub, 1 July 2026. [Online]. Available: https://github.com/RoyalHollowayMakersSoc/E01_ESP-NOW_Message_Comms. [Accessed 1 July 2026].

Author: Henry Li, (Secretary, 24-27)

First published: Friday, 03 July 2026

Last revised by: Henry Li, (Secretary, 24-27)

Last revised: Friday, 03 July 2026

Revision: 2